INTERNAL BUILD DOCUMENTATION

Comfy Agent

A meta-node that reads natural language and builds ComfyUI workflows directly on the canvas — how it's put together, and how it actually came to work.

HOST — ComfyUI Desktop | **PRIMARY** — qwen3.5:0.8b | **FALLBACK** — qwen3.5:9b | **VERIFIED** — live, 1,717 nodes

---

§01 What this is

Comfy Agent is not a sidebar, not an external tool, and not a chatbot that hands you JSON to paste in. It is a ComfyUI custom node — `ComfyAgent` — that lives on the canvas like any other node, and when you click its **Build Workflow** button, it draws new nodes and wires connections around itself in real time. Its output is a workflow, not image data.

It does this in three stages, corresponding to three jobs:

JOB 1	JOB 2	JOB 3
Understand	**Reason**	**Build**
Turn a natural-language task_prompt into a workflow intent.	Decide which nodes are needed, how they connect, and their widget values — validated against the live install first.	Draw the result on canvas without overlap, wire the links, report status.

The generation strategy is **blueprint-first**: an 8-template library covers the common cases (txt2img, img2img, upscale, ControlNet, LoRA stacking, face detailing, video) and is tried before anything else, because it's instant and never hallucinates. Free-form generation — a four-stage LLM pipeline that can build a workflow from scratch — is the fallback path for requests no blueprint matches.

§02 Why this is harder than it looks

Manually crafting a ComfyUI workflow requires knowing which of 1,700+ installed node types apply, what their exact input/output names are, and how their types chain together — none of which an LLM should be trusted to recall from memory. Two decisions follow from that:

- The live `/object_info` endpoint is the **only source of truth** for node shapes. No node's inputs, outputs, or types are ever answered from a model's memory.
- **No LLM ever emits raw workflow JSON.** It either selects a pre-validated blueprint and fills in the details, or its output is decomposed into four narrow, independently-validated stages — never one big “write me a workflow” call.

The primary model is `qwen3.5:0.8b` — chosen for being already installed and fast, but small enough that every design decision in this document exists specifically to keep it from being asked more than it can reliably do in one call.

§03 Architecture at a glance

SIGNAL PATH

```
 ComfyUI (browser)                          ComfyUI (Python backend)
+---------------------+                    +--------------------------+
|  ComfyAgent node    |  POST /agent/      |  agent_routes.py         |
|  (task_prompt, mode, |  build_workflow    |   |- blueprint_matcher   |
|   Build Workflow ->) |------------------->|   |   (instant, no LLM)  |
|                     |                    |   `- generation/pipeline |
|  agent_node_ui.js   |<-------------------|       (4-stage, Ollama)  |
|  canvas_writer.js   |  workflow_json +   |                          |
|  (draws the result)  |  live WS progress  |  llm_client.py --------->|--> Ollama
+---------------------+                    +--------------------------+   (localhost:11434)
```

The JS side never talks to Ollama or the node registry directly — it only calls the one `/agent/build_workflow` route and draws whatever `workflow_json` comes back. All reasoning happens server-side in Python, which is what makes every stage independently testable — and, as it turned out, independently debuggable without a browser in the loop at all.

§04 Package structure

```
custom_nodes/comfy_agent/

|-- __init__.py              registers via comfy_entrypoint, sets WEB_DIRECTORY
|-- agent_node.py            the ComfyAgent node itself (V3 schema)
|-- agent_routes.py          /agent/ping, /agent/build_workflow
|-- llm_client.py            async Ollama client, retry + fallback escalation
|-- config.py                every model name / timeout / budget, in one place
|-- knowledge/
|   |-- schema_compactor.py  /object_info fetch + category filter + token budget
|   |-- type_validator.py    type-safe link validation (case-normalized)
|   `-- blueprint_matcher.py keyword -> blueprint selection + loading
|-- generation/
|   |-- classify.py          Pass 1 -- which schema categories does this need
|   |-- stage_a_identify.py  Stage A -- which node types
|   |-- stage_b_configure.py Stage B -- widget values per node
|   |-- stage_c_connect.py   Stage C -- links between nodes
|   |-- stage_d_validate.py  Stage D -- structural check before it ships
|   `-- pipeline.py          orchestrates Pass 1 + Stages A-D, streams progress
|-- blueprint_defs/          8 pre-authored workflow templates (JSON)
`-- js/
    |-- agent_node_ui.js     Build button, status widget, WS progress listener
    |-- canvas_writer.js     overlap-aware placement, node/link injection
    `-- progress_listener.js WebSocket -> status widget wiring
```

§05 Request lifecycle

Blueprint path — the fast, reliable path

```
SEQUENCE — BLUEPRINT PATH

 1.  User                ->  canvas_writer.js     click "Build Workflow"
 2.  canvas_writer.js    ->  agent_routes.py      POST /agent/build_workflow
 3.  agent_routes.py     ->  blueprint_matcher    match(task_prompt) -- keyword score, no LLM
 4.  blueprint_matcher   -->  agent_routes.py      "txt2img_sdxl"
 5.  agent_routes.py     ->  blueprint_matcher    load_blueprint("txt2img_sdxl")
 6.  blueprint_matcher   -->  agent_routes.py      workflow JSON (nodes + links)
 7.  agent_routes.py     -->  canvas_writer.js     workflow_json, execution_ready: true
 8.  canvas_writer.js    ->  canvas_writer.js     injectWorkflow() -- place, wire, pan view
```

No Ollama call happens on this path at all — it's why blueprint mode works even with Ollama offline, and why it's instant.

Free-form path — the fallback, when no blueprint matches

```
SEQUENCE — FREE-FORM PATH

 1.  agent_routes.py     ->  pipeline.py          generate_workflow(task_prompt)
 2.  pipeline.py         ->  schema_compactor     fetch_object_info() -- 1700+ live node defs
 3.  pipeline.py         ->  Ollama               Pass 1 -- classify_task() -> schema categories
 4.  pipeline.py         ->  schema_compactor     compact_schema() -- filter + token budget
 5.  pipeline.py         ->  Ollama               Stage A -- identify_nodes() -> node types
 6.  pipeline.py         ->  Ollama               Stage B -- configure_widgets() per node
 7.  pipeline.py         ->  Ollama               Stage C -- generate_connections() -> links
 8.  pipeline.py         ->  pipeline.py          Stage D -- validate_spec() structural check
 9.  pipeline.py         -->  agent_routes.py      node_instances, widgets, links
```

Every stage's LLM output is treated as a *proposal*: Stage A's node names are checked against the real `/object_info` keys, Stage C's links are independently re-validated by `type_validator.py`, and Stage D can still reject the whole thing before it ever reaches the browser.

§06 The knowledge layer

schema_compactor.py

Turns the live `/object_info` payload — 1,717 nodes on this install — into something a small model can actually hold in context. It buckets every node into categories (loaders, sampling, latent, conditioning, image, upscaling, controlnet, video, mask, custom) by keyword match, filters to only the categories a request needs, and caps the result by an estimated character budget tied to `config.PASS2_MAX_SCHEMA_TOKENS` — not a raw node count, since entries vary wildly in size.

type_validator.py

Normalizes every type string to uppercase before comparing (`"model"` → `"MODEL"`), so link validation never breaks on a case mismatch between what an LLM emits and what `/object_info` reports. It's the single gate every proposed link passes through before acceptance.

blueprint_matcher.py

Scores a task_prompt against each blueprint's trigger keywords and picks the best match. Blueprint files live in `blueprint_defs/` and are loaded directly by Python file I/O — deliberately *not* through ComfyUI's own

native “Global Subgraph” feature, which expects a different, undocumented internal format.

§07 The four-stage pipeline

Single-pass “write me a whole workflow” generation is where small models fail hardest, so free-form generation is split into four narrow calls, each asking for the least possible output.

STAGE	ASKS THE MODEL FOR	EXPLICITLY EXCLUDES
`A -- Identify`	`a bare JSON array of node class_type names`	widgets, links
`B -- Configure`	`one node's widget values, one call per node`	every other node in scope
`C -- Connect`	`links between already-validated instances`	widget concerns
`D -- Validate`	`(no LLM call) -- structural check`	generation entirely

Every stage uses Ollama's `format` parameter — a JSON-schema grammar constraint, not a hinted instruction — so the output is structurally forced to match, not just asked nicely to.

Escalation: when “valid JSON” still isn't a good answer

The retry logic only escalates from the primary model to the fallback when a call fails outright — bad HTTP, unparseable JSON. But a small model's most common failure mode isn't a crash: it's returning perfectly valid JSON that's still wrong — two nodes when six are needed, a hallucinated widget value, a link with the whole input list copied in as one string.

Stages A and C each carry an extra check for exactly this: if the primary model's *validated* result is implausibly sparse, the pipeline silently tries the fallback model once more and keeps whichever result is more complete. This — not the retry-on-error path — is what actually made free-form generation reliable.

§08 The frontend

`agent_node_ui.js` renders the Build Workflow button and a read-only status widget, wired to a WebSocket listener so status updates live as the backend streams progress: *“Fetching node registry…”* → *“Identifying nodes…”* → *“Done — built N nodes, M links.”*

`canvas_writer.js` is where the drawing happens:

- `computeSpecFootprint()` measures how much canvas space the spec actually needs, so `findFreeZone()` checks the *whole* footprint against existing nodes — not a fixed placeholder box a large blueprint could still overlap.
- `injectBlueprint()` / `injectFreeForm()` call `LiteGraph.createNode()` per node inside a try/catch, so one failed node type never takes down the whole build.

- `maybeWrapSubgraph()` attempts to fold builds over 6 nodes into a SubgraphNode, falling back to a flat list if the wrap throws — caught and handled, not a blocker.
- After injection, the view pans to the new nodes via `centerOnNode()` — without this, a successful build could land entirely outside the viewport and look identical to a silent failure.

§09 What broke, and what fixed it

This section only exists because the thing was actually run against a live ComfyUI Desktop install and local Ollama — not from reading the code. Ordered by how deep in the stack each one was hiding.

LOADER `__init__.py`

The loader never ran our V3 node

`nodes.py`'s custom-node loader checks for `NODE_CLASS_MAPPINGS` before `comfy_entrypoint`, and short-circuits the moment it finds one — even an empty dict, since the check is `is not None`, not truthiness.

Fix — export `comfy_entrypoint` only.

FRONTEND `canvas_writer.js`

A successful build looked identical to a silent failure

`findFreeZone` checked a fixed 300×200 box no matter how many nodes the spec contained, and nothing panned the viewport afterward — a 7-node blueprint could land far outside the visible canvas with zero visible change.

Fix — size the free-zone check to the real spec footprint; auto-pan to the result.

CRITICAL `llm_client.py`

Ollama's thinking models silently ate every free-form response

`qwen3.5:0.8b` is a “thinking” model — with `format` schema constraints on, it wrote the actual JSON into Ollama's separate `thinking` field and left `response` empty. Every single free-form call failed with “not valid JSON: ” (an empty string), regardless of prompt quality — this looked like everything was broken.

Fix — send `think: false`; parse `thinking` as a defensive fallback.

ROOT CAUSE `classify.py`

Pass 1 classification silently starved the candidate pool

Direct testing showed the classifier regularly returned `['sampling']` only for a plain txt2img request — dropping `loaders` and `conditioning` entirely. `CheckpointLoaderSimple` and `CLIPTextEncode` were never even in the list Stage A picked from, no matter how good Stage A's own prompt was. Found only by logging Pass 1's raw output across repeated runs.

Fix — always union the classifier's answer with a fixed core category set; Pass 1 only adds specialized categories on top.

DESIGN GAP `stage_c_connect.py`

Stage C was connecting nodes blind

Its prompt only ever listed `instance_id: class_type` — the `schema` parameter it received was used only to validate the model's answer afterward, never shown to the model beforehand. It had no way to know KSampler's inputs were even called model/positive/negative/latent_image.

Fix — the prompt now lists each instance's real input/output names plus a worked example link.

VALIDATION `stage_d_validate.py`

A broken result could still report success

Stage D collected “orphan node” warnings into an errors list, but the `valid` flag never actually checked it — a completely disconnected multi-node result (0 links between 6 nodes) reported `execution_ready: true`.

Fix — a fully disconnected multi-node graph now fails validation outright.

VALIDATION `stage_b_configure.py`

Hallucinated widget values would have broken on canvas

Type-coercion only handled INT/FLOAT/BOOLEAN — COMBO widgets like `sampler_name` (a fixed enum, not free text) passed straight through unchecked, so a value like `"SDXL"` would have been written directly onto a real node.

Fix — validate COMBO values against the node's real allowed options; tell the model what those options are instead of a bare widget name.

DEAD CONFIG `schema_compactor.py`

A budget that was declared but never enforced

`PASS2_MAX_SCHEMA_TOKENS` existed in `config.py` from the start but nothing enforced it — the real candidate pool for a common category set measured ~3,300 tokens, already past the declared 2,500-token budget, before the task prompt was even added.

Fix — tie `compact_schema`'s truncation to that budget for real.

ENVIRONMENT `blueprint_defs/`

ComfyUI's own subgraph feature, misapplied

Naming the blueprint folder `subgraphs/` made ComfyUI's frontend try to auto-discover and load them as native “Global Subgraphs” on every page load — a feature this project doesn't use, since blueprints are read directly by Python. That produced 8 console errors every session.

Fix — renamed to `blueprint_defs/`, which nothing else auto-scans.

ENVIRONMENT `config.py`

A fallback model that didn't exist

`FALLBACK_MODEL` was set to `llama3.1:8b`, never pulled on this machine's Ollama — any escalation attempt 404'd instead of retrying on a working model.

Fix — point at `qwen3.5:9b`, which was actually installed.

Each of these was found the same way: run the actual thing, read the actual error, trace it to its actual cause — not guessed from the code.

§10 Known limitation, honestly stated

Free-form generation quality is bounded by the 0.8B primary model's reasoning, exactly as anticipated going in. Even after every fix above, Stage C sometimes still leaves one plausible link unconnected (most often VAEDecode → SaveImage) on some runs — the escalation logic catches the *worst* failures but doesn't guarantee a fully complete graph on every call. This isn't a bug to chase further; it's the measured ceiling of a sub-1B model, and it's exactly why blueprint mode — which has no such ceiling, since it's pre-authored — is the primary path, not the fallback.

§11 Configuration reference

All tunable values live in `config.py`.

SETTING	VALUE	WHY
`PRIMARY_MODEL`	`qwen3.5:0.8b`	fast, already installed, low resource footprint
`FALLBACK_MODEL`	`qwen3.5:9b`	escalation target when the primary's output is invalid or sparse
`STAGE_RETRY_CAP`	`1`	same-model retries before escalating
`PASS2_MAX_SCHEMA_TOKENS`	`2500`	schema block budget handed to Stage A
`SUBGRAPH_WRAP_THRESHOLD`	`6`	node count above which a build attempts subgraph wrapping

SETTING	VALUE	WHY
`OLLAMA_GENERATE_TIMEOUT_SECONDS`	`60`	per-call timeout
`ALLOW_NETWORK`	`False`	routes reject non-localhost requests by default

§12 Extending it

Add a blueprint

Build the workflow in ComfyUI's own canvas, export it as workflow JSON, drop it in `blueprint_defs/<name>.json`, and add its trigger keywords to `BLUEPRINT_KEYWORDS` in `blueprint_matcher.py`. No Python changes needed.

Add a mode

MODIFY/EXPLAIN/DEBUG/SUGGEST are wired into `agent_routes.py`'s dispatch but currently return “not implemented yet” — each needs a handler following the same shape as `_handle_build`, reading the current canvas state instead of building from scratch.